P10 Death and Respawn

CRE132 worksheet

What you'll make

Dying that doesn't end the game. When the player dies on the spikes or falls into a pit, the hero drops down dead, then comes back at the last **checkpoint** with full health and one life fewer. Touch a dark stone and it lights up: that's your new checkpoint. With no lives left, the level starts again from the beginning with 3 lives.

You'll change two scripts you already have (**Health** and **GameManager**), and see why one death sequence for every way of dying keeps a game consistent.

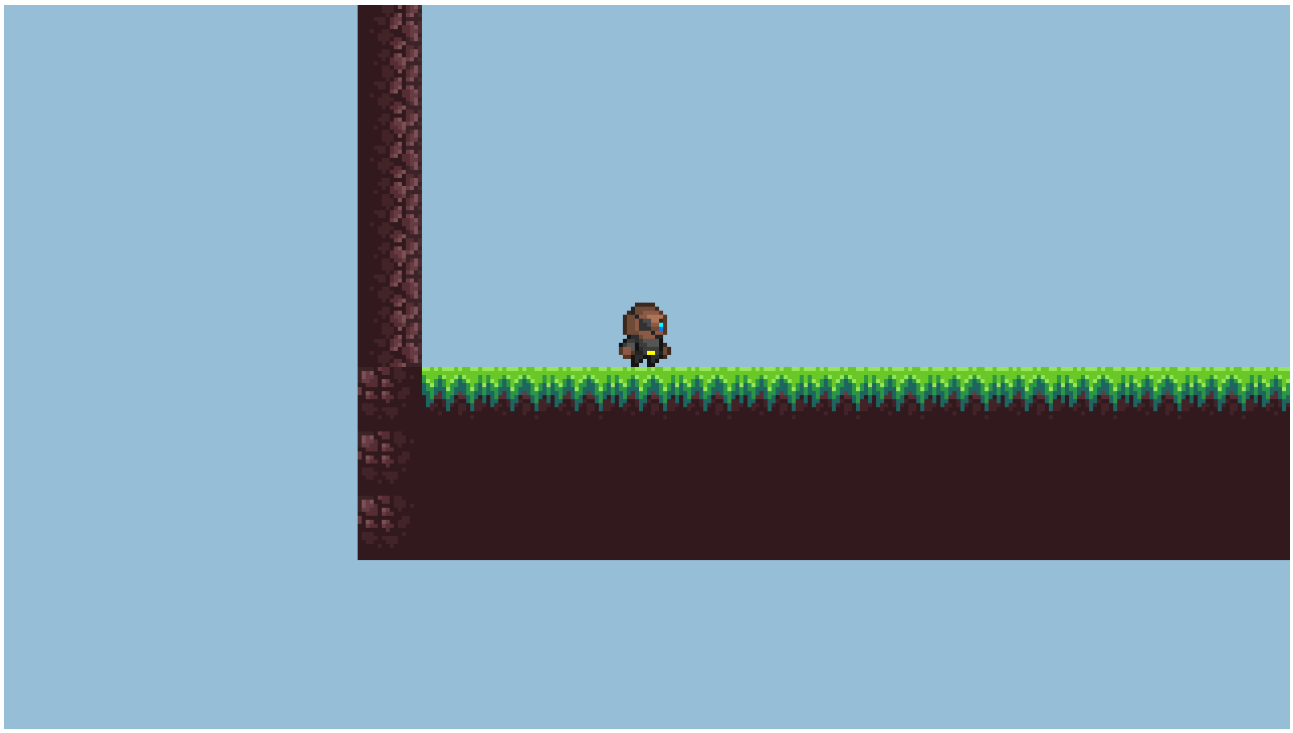


Figure 1: The start of the level: the player by the left wall, with the pits, spikes and checkpoints further along.

Before you start

1. Unzip `CRE132_P10_DeathAndRespawn_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P10_DeathAndRespawn_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P10_DeathAndRespawn`.
3. Press **Play**. You can run (A/D or arrows) and jump (Space). The health bar shows 3 and the top right says **Lives: 3**. The **Spikes** hurt (S01's `DamageZone`, 1 a second). Lose all your health and the player is simply destroyed. Fall into a pit and you fall for ever. Press Play again to stop.

The scene also has three **Checkpoint** stones (in **Checkpoints**) with trigger colliders and a glow animation, a **Fall Death Zone** trigger under the level, and the **GameManager** prefab. The player's Animator already has a **Dead** state that plays when the **Die** trigger is set.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Health that doesn't destroy the player

Open `Assets/_game/Scripts/Health/Health.cs`. Below the `maxHealth` line, add a new setting:

```
    // Untick for a player who respawns (P10): the object stays, and something else
    // handles death.
    [SerializeField] private bool destroyOnDeath = true;
```

Then find the `Die` method near the bottom and replace it with this, which also adds a new `ResetHealth` method below it:

```
    private void Die()
    {
        Debug.Log(gameObject.name + " died");
        onDied.Invoke();
        if (destroyOnDeath)
        {
            Destroy(gameObject);
        }
    }

    // Back to full health, for respawning.
    public void ResetHealth()
    {
        currentHealth = maxHealth;
        SendHealthChanged();
    }
```

- **`destroyOnDeath`** is ticked by default, so enemies and dummies are still destroyed as before. For the player you untick it: the object stays, and another script decides what dying means.
- **`onDied.Invoke()`** still runs either way. That's the S09 event you'll use to start the death sequence.
- **`ResetHealth`** puts health back to full and calls `SendHealthChanged`, so the health bar fills up again too.

Add it: save, then select the **Player** and check that **Destroy On Death** on its Health is **unticked**.

2. Lives in the GameManager

Open `Assets/_game/Scripts/Managers/GameManager.cs`. Below the `highScore` line, add:

```
    // Lives, for games where the player respawns (P10).
    [SerializeField] private int startingLives = 3;
    private int lives;

    public int Lives
    {
        get { return lives; }
    }
```

At the end of `Awake`, after the `highScore = PlayerPrefs...` line, add the last line shown here:

```
        // 0 if nothing has been saved yet.
        highScore = PlayerPrefs.GetInt("HighScore", 0);
```

```
        lives = startingLives;
    }
```

Then add two new methods below `Awake`:

```
    public void LoseLife()
    {
        lives = lives - 1;
        Debug.Log("Lives left: " + lives);
    }

    public void ResetLives()
    {
        lives = startingLives;
    }
```

- **Lives belong to the GameManager**, not the player. The GameManager survives scene reloads (S02's `DontDestroyOnLoad`), so lives last longer than any one player object.
- **`Lives` is read-only** from outside, like Health's `CurrentHealth`. Other scripts change it only through `LoseLife` and `ResetLives`.

Test it: save and check the Console has no red errors. Select the **GameManager** in the Hierarchy: **Starting Lives** is 3.

3. The death sequence: PlayerRespawn

Make a folder `Assets/_game/Scripts/P10_DeathAndRespawn` and create `PlayerRespawn` in it:

```
// What happens when the player dies: play the death animation, lose a life, then come
// back at the last checkpoint with full health. With no lives left the level starts again
// from the beginning. Health's On Died event calls Die(). Health's Destroy On Death is
// unticked, so the player isn't destroyed.
using System.Collections;
using UnityEngine;
using UnityEngine.SceneManagement;

public class PlayerRespawn : MonoBehaviour
{
    [SerializeField] private float deathTime = 1.2f;

    private Vector3 respawnPosition;
    private Health health;
    private Rigidbody2D body;
    private Animator animator;
    private BetterJumpController controller;
    private PlatformerAnimator platformerAnimator;

    private void Awake()
    {
        health = GetComponent<Health>();
        body = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
        controller = GetComponent<BetterJumpController>();
        platformerAnimator = GetComponent<PlatformerAnimator>();

        // Until a checkpoint is reached, come back where the level started.
        respawnPosition = transform.position;
    }
```

```
    public void SetCheckpoint(Vector3 position)
    {
        respawnPosition = position;
        Debug.Log("Checkpoint reached");
    }

    public void Die()
    {
        StartCoroutine(DeathRoutine());
    }

    private IEnumerator DeathRoutine()
    {
        // Stop moving and controlling, and play the Dead animation (Any State → Dead on
        // the Die trigger).
        controller.enabled = false;
        platformerAnimator.enabled = false;
        body.linearVelocity = Vector2.zero;
        body.simulated = false;
        // Tell the Animator we're grounded and still, or Any State → Fall could take over
        // from Dead.
        animator.SetBool("IsGrounded", true);
        animator.SetFloat("VerticalSpeed", 0f);
        animator.SetFloat("Speed", 0f);
        animator.SetTrigger("Die");

        yield return new WaitForSeconds(deathTime);

        GameManager.Instance.LoseLife();
        if (GameManager.Instance.Lives <= 0)
        {
            // Game over: full lives and score again, and back to the very start.
            GameManager.Instance.ResetLives();
            GameManager.Instance.ResetScore();
            SceneManager.LoadScene(SceneManager.GetActiveScene().name);
            yield break;
        }

        // Back to the checkpoint, alive again.
        transform.position = respawnPosition;
        body.simulated = true;
        health.ResetHealth();
        animator.Play("Idle");
        controller.enabled = true;
        platformerAnimator.enabled = true;
    }
}
```

What's going on:

- **`respawnPosition`** starts as wherever the player is when the level starts. A checkpoint changes it with `SetCheckpoint`.
- **`Die` starts a coroutine**, so the sequence can wait for the death animation with `WaitForSeconds(deathTime)`.
- **Freeze first.** The controller and `PlatformerAnimator` are switched off, and `body.simulated = false` takes the body out of physics, so nothing moves the player during the animation. Setting the Animator's **IsGrounded**, **VerticalSpeed** and **Speed** stops the **Any State > Fall** transition taking over from **Dead** in mid-air.

- **Lives left:** move to the checkpoint, switch physics back on, reset health, jump the Animator straight to **Idle** with `animator.Play`, and give control back. **No lives:** reset lives and score and reload the scene. `yield break` ends the coroutine there.

Add it: select the **Player** and **Add Component > Player Respawn**. Leave **Death Time** at 1.2. Then, on the Player's **Health**, find **On Died ()**. The list is empty: click **+** to add a row, drag the **Player** from the Hierarchy into the row's object slot, and choose **PlayerRespawn > Die ()** from the function list.

Test it: stand in the spikes until your health runs out. The hero falls down dead, then comes back at the start with full health. The Console says **Lives left: 2**.

4. Checkpoints

Create `Checkpoint` in the `P10_DeathAndRespawn` folder:

```
// Touch the stone to make it light up and become the place the player comes back to after
// dying.
using UnityEngine;

public class Checkpoint : MonoBehaviour
{
    private SpriteRenderer spriteRenderer;
    private Animator animator;
    private bool isActive;

    private void Awake()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        animator = GetComponent<Animator>();

        // Dark and still until it's reached.
        spriteRenderer.color = new Color(0.5f, 0.5f, 0.5f);
        animator.enabled = false;
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (isActive)
        {
            return;
        }

        PlayerRespawn player = other.GetComponent<PlayerRespawn>();
        if (player != null)
        {
            isActive = true;
            player.SetCheckpoint(transform.position);
            spriteRenderer.color = Color.white;
            animator.enabled = true;
        }
    }
}
```

- **`Awake` makes the stone dark and still**: a grey tint and the Animator switched off.
- **On first touch** by something with `PlayerRespawn`, it stores its own position as the respawn point, turns white and starts its glow animation. **`isActive`** makes sure that only happens once.

Add it: select all three **Checkpoint** objects (in **Checkpoints**) and add **Checkpoint**. Their Box Collider 2D already has **Is Trigger** ticked.

Test it: walk past the first stone and it lights up. Die on the spikes and you come back at that stone, not at the start.

5. Pits: FallDeath

Create `FallDeath` in the same folder:

```
// A trigger under the level: falling into a pit takes all the player's health, so the
// death sequence runs.
using UnityEngine;

public class FallDeath : MonoBehaviour
{
    private void OnTriggerEnter2D(Collider2D other)
    {
        Health health = other.GetComponent<Health>();
        if (health != null && other.CompareTag("Player"))
        {
            health.TakeDamage(health.CurrentHealth);
        }
    }
}
```

- **`TakeDamage(health.CurrentHealth)`** takes all the health that's left. Falling uses exactly the same death sequence as the spikes, with no extra code.
- **`CompareTag("Player")`** means only the player dies this way.

Add it to **Fall Death Zone**, the long trigger under the level.

Test it: jump into a pit. You die and come back at the last checkpoint.

6. Lives on screen: LivesDisplay

Create `LivesDisplay` in `Assets/_game/Scripts/UI`:

```
// Shows the GameManager's lives ("Lives: 3"), changing the text only when the number
// changes.
using TMPro;
using UnityEngine;

public class LivesDisplay : MonoBehaviour
{
    [SerializeField] private TMP_Text livesText;

    private int shownLives = -1;

    private void Update()
    {
        if (GameManager.Instance == null)
        {
            return;
        }

        int lives = GameManager.Instance.Lives;
        if (lives != shownLives)
        {
            shownLives = lives;
            livesText.text = "Lives: " + lives;
        }
```

```
    }
}
```

- **`shownLives`** remembers the number on screen, so the text only changes when the lives do. It starts at −1 so the first frame always updates it.
- **The `null` check** skips the frame if there's no GameManager yet.

Add it: select **Lives Panel** (in **HUD Canvas**) and add **Lives Display**. Drag its child **Text** into the **Lives Text** slot.

Test it: die three times. The lives go 3, 2, 1, then the level starts again from the very beginning with **Lives: 3**.

Check your work

- ☐ Dying on the spikes or in a pit plays the death animation, then you come back with full health.
- ☐ Each death takes one life, and the lives text changes.
- ☐ Touching a stone lights it up, and you come back at the last stone you touched.
- ☐ With no lives left, the level restarts from the beginning with 3 lives.
- ☐ Compare with `CRE132_P10_DeathAndRespawn_Finished` if something behaves differently.

Key points

- **One death sequence for every way of dying.** Spikes and pits both just take health; `PlayerRespawn` handles what happens next.
- **Lives belong to the GameManager** (they outlast the player and survive reloads). **The respawn point belongs to the player.**
- **`body.simulated = false`** freezes a Rigidbody 2D completely, so nothing pushes it while the death animation plays.

If something goes wrong

- **The player disappears when it dies: Destroy On Death** is still ticked on the Player's Health.
- **The player dies but never comes back: On Died** has no **PlayerRespawn > Die** in it, or the object slot is empty.
- **NullReferenceException in PlayerRespawn:** there's no **GameManager** in the scene, or PlayerRespawn isn't on the Player next to its controller and animator scripts.
- **The Fall animation plays instead of the Dead one:** the three `animator.Set...` lines before `SetTrigger("Die")` are missing or misspelt. Parameter names must match the Animator exactly.
- **Checkpoints don't light up:** Checkpoint looks for **PlayerRespawn**, so add that to the Player first.
- **The lives text never changes:** the **Lives Text** slot is empty.

Try it

1. Show a "Game Over" panel (S06) instead of restarting straight away.
2. Add a 1-up pickup that calls a new `GameManager.AddLife()` method.
3. Make the camera hold still during the death animation.